Transparent Space UI Design Audit

A structured review of the visual language, information architecture, dashboards, and data-visualization layer of the Transparent Space frontend — including what is working well today, what is friction or risk, and a prioritized list of changes that would compound the most.

MARKETS MADE VISIBLE // FRONTEND // NEXT.JS 15 · TAILWIND V4 · VISX

Repository
BrunoEleodoro / transparent-space

Surface reviewed
app/, components/, app/globals.css

Report date
2026-04-07

Method
Static code audit of pages, components, design tokens, and chart layer

Out of scope
Live perf, SEO, backend data quality

Audience
Founders, design, frontend

01 Executive summary

Transparent Space already has the backbone of a **premium, opinionated analytics product**: a coherent "blueprint" visual identity, a three-tier typographic system (JetBrains Mono / Inter / IBM Plex Mono), a serious chart catalog, and a multi-tenant space model with login, sidebar navigation, and pool-level dashboards. The product *looks the part* for a market-microstructure tool aimed at LPs, market makers, and operators.

The biggest gaps are not visual taste — they are **systemic consistency**. Design tokens exist in CSS but are bypassed by components that hardcode hex values; charts use three different rendering libraries with three different color and formatting conventions; loading, empty, and error states are present in some places and silently missing in others; and several high-traffic surfaces (sidebar FAB, RFQ tab nesting, pool header on mobile) have UX edge cases that will surface as the user base widens.

None of this is a rewrite. Most of it is a **2–4 sprint hardening pass** around four pillars: *tokens*, *states*, *charts*, and *navigation*.

Scorecard

VISUAL IDENTITY
8 / 10
Strong, cohesive blueprint look. Premium feel.

DESIGN SYSTEM
5 / 10
Tokens exist; components don't consistently use them.

INFORMATION ARCHITECTURE
6 / 10
Clear at top, gets foggy in nested RFQ flows.

DATA VIZ QUALITY
6 / 10
Best charts are excellent. Worst feel hand-rolled.

STATES & RESILIENCE
5 / 10
Skeletons exist; empty/error coverage is uneven.

ACCESSIBILITY
4 / 10
Some ARIA, weak focus, small touch targets.

RESPONSIVE
5 / 10
Breakpoints used; key flows still desktop-first.

OVERALL
6.0 / 10
Strong foundation, scattered execution.

ONE-LINE VERDICT

Transparent Space is a **visually confident product with a half-finished design system underneath**. The fastest way to compound is to harden tokens, unify the chart layer, and finish the "quiet" states (empty, error, loading) — not to redesign anything.

02 How this review was conducted

This audit is a **static code review** of the live frontend in `/transparent-space`. No design files, prototypes, or analytics were used — only the source of truth that ships to users.

- **Surface inspected:** `app/` (routes, layout, providers), `components/` (sidebar, dashboard, RFQ, birds-eye, auth, ui primitives), `app/globals.css` (design tokens), `components.json`.
- **Lenses applied:** visual identity, design tokens, primitive components, page composition, information architecture, data visualization, loading / empty / error states, accessibility, responsiveness.
- **Citations:** findings reference `file:line` wherever possible so each claim can be verified and acted on.
- **Out of scope:** Lighthouse / runtime performance, SEO, backend data correctness, copywriting, brand strategy. These are worth a separate pass.

03 Visual identity & brand language

What's working

STRONG "BLUEPRINT" AESTHETIC

- Cohesive technical look: subtle grids, corner brackets on cards, monospaced titles. Reads as serious infrastructure, not a generic SaaS template.
- Three-font system (`JetBrains Mono` for titles, `Inter` for body, `IBM Plex Mono` with `tabular-nums` for numbers) is the right call for a numbers-heavy tool. `app/globals.css:63–66`
- OKLch color tokens for both light and dark themes are defined cleanly. `app/globals.css:46–171`

CARD PRIMITIVE HAS PERSONALITY

- `.bp-card` defines consistent 20 px padding, soft 1 px border, and decorative corner markers via `::before`/`::after`. `app/globals.css:222–290`
- Hover state is subtle (shadow only, no jump) — appropriate for a dense dashboard.
- Typographic hierarchy inside cards is well-tuned: 11 px uppercase labels, 28 px metric values, tabular alignment.

What's weak

TOKENS ARE BYPASSED BY COMPONENTS

- Several components hardcode hex values instead of consuming the `--bp-*` CSS variables, e.g. `sidebar.tsx:445` uses `dark:bg-[#1a1a2e]/90`; `space-loader.tsx:79` uses `bg-white dark:bg-[#0a0a0f]`.
- This means a future palette change has to be done in dozens of places, not one.

NO TAILWIND CONFIG EXTENDING THE BLUEPRINT TOKENS

- The `--bp-*` tokens are **only** usable as raw CSS variables. There is no `bg-bp-card`, `text-bp-accent`, etc. exposed to Tailwind utilities.
- Result: new code defaults to arbitrary `[]` Tailwind values, which is exactly how drift starts.

TWO PARALLEL THEME SYSTEMS

- shadcn's OKLch tokens (`--background`, `--card`, `--primary`…) live alongside the custom `--bp-*` system. They overlap in meaning but not in value.
- Pick one as the source of truth and have the other one alias into it.

HELP MODAL CARRIES HARDCODED SVG ILLUSTRATIONS

- `components/ui/help-modal.tsx` inlines 200+ lines of SVG with hardcoded indigo (`#6366f1`). It will not respond to theme changes and is hard to maintain.

04 Information architecture & navigation

The product's top-level model is sound: *auth* → *space* → *pool* → *dashboard view*, with sidebar collapse driven by Framer Motion and clean URL sync via `PoolUrlSync` and `spaceSlug` routing (`app/[spaceSlug]/page.tsx`). Things start to get foggy *inside* a dashboard, particularly RFQ.

What's working

- **Multi-tenant routing is clean.** A user with multiple spaces is redirected to `/{slug}` after login and the URL is the source of truth for which space is active. `app/page.tsx:33–42`, `app/[spaceSlug]/page.tsx:79–100`
- **Sidebar groups pools by asset** with a curated `ASSET_ORDER`, which is the right grouping for a market-maker audience.
- **Animated sidebar collapse** uses a tasteful Framer easing curve and the rest of the dashboard reflows around it via `DashboardWrapper`. Feels considered, not janky. `components/dashboard/dashboard-wrapper.tsx:40–46`
- **Empty space dashboard** handles the "no pools yet" case with a friendly avatar & copy block. `app/[spaceSlug]/page.tsx:177–193`

What's weak

1. THE SIDEBAR FAB IS THE SINGLE HIGHEST-FRICTION ELEMENT

`components/sidebar/floating-action-button.tsx` is rendered absolutely with `z-20` and overlaps the dashboard canvas when the sidebar is collapsed. On narrow viewports it hovers *on top of* chart content. It is also the only way to re-open the sidebar, so users cannot ignore it. Either anchor it inside the sidebar rail or move it into the `DashboardWrapper` layout flow with proper offset.

2. RFQ HAS TAB NESTING WITHOUT CONTEXT

The RFQ subtree contains seven dashboards (`rfq-depth`, `rfq-cowswap`, `rfq-bebop`, `rfq-0x`, `rfq-1inch`, `rfq-jupz`, plus the historical/overview tabs in `rfq-control-panel.tsx:71-78`). Once a user is two clicks deep there is no breadcrumb, no "you are here" affordance, and no way back to the parent pool except via the sidebar. This is fixable with a small persistent breadcrumb in `rfq-header.tsx`.

3. SEARCH BAR DISAPPEARS, NOT JUST COLLAPSES

`components/sidebar/search-bar.tsx:28-40` animates the input width to `40 px` when the sidebar collapses, leaving only an icon. The icon is the same size as other icon buttons, so users can't tell "this is a search field" from "this is a button." A magnifying glass cursor or a ghost placeholder fixes it.

4. POOL NAME OVERFLOW ON NARROW WIDTHS

`components/dashboard/pool-header.tsx:47` renders the pool name with no truncation. Long pairs (e.g. `cbBTC / USDC 0.05% (CL)`) wrap awkwardly on tablet. Add `truncate` + a tooltip on hover.

5. LOADER TRANSITIONS ARE TOO LONG FOR POWER USERS

`app/[spaceSlug]/page.tsx:32-33` hardcodes both `INITIAL_TRANSITION_MS = 1500` and `POOL_SWITCH_TRANSITION_MS = 1500`. A 1.5 second forced wait on every pool switch is a great first-impression demo, but a usability tax for an LP who clicks 30 pools an hour. Consider 1500 ms on first load and 200–400 ms on subsequent switches, or skip entirely once data is in cache.

05 Pages & primary flows

Login & auth

STRENGTHS

- Clean centered card layout with good vertical hierarchy.
- Multi-step state machine handles password change and space selection in the same surface (`app/page.tsx`).
- Error block is well-styled with semantic red. `components/auth/login.tsx:211–231`

WEAKNESSES

- `disabled` and `isLoading` render the same visual on the submit button (`login.tsx:193–197`) — a user can't tell if the form is invalid or just thinking.
- No "forgot password" affordance even though the system supports password change.
- Logo / brand mark on the login screen could lean into the blueprint look much harder than it currently does.

Pool dashboard (default view)

`components/dashboard/dashboard.tsx` is the highest-traffic surface. It uses a `grid-cols-1 sm:grid-cols-2 lg:grid-cols-6` KPI strip on top (`dashboard.tsx:1710`) and chart rows below.

STRENGTHS

- Density is appropriate for the audience — this is a Bloomberg-flavored tool, not a retail dashboard.
- KPI strip uses tabular numerics so values align column-to-column even when widths differ.
- Tabs (`aria-label="Dashboard tabs"` at `dashboard.tsx:1806`) cleanly separate *Pools* and *RFQ* at the top of the analysis surface.

WEAKNESSES

- No `max-w-*` on the dashboard canvas. On 4K and ultra-wide displays the charts stretch unbounded and become hard to read.
- KPI cards are driven by child content height, which causes uneven row heights when one card has a delta and another doesn't. `components/dashboard/metric-card.tsx:14–24`
- No global error boundary — one chart that throws can take down the dashboard tree.
- No global time-range picker. Each chart owns its own viewport, so "show me everything in the last 24h" is many clicks.

Birds-eye & token birds-eye

The birds-eye family (`components/dashboard/birds-eye/` and `components/dashboard/token-birds-eye/`) is the most opinionated part of the product. The flip-card / table toggle pattern is a great idea.

- **Tab toggle is below the touch target threshold.** `birds-eye-dashboard.tsx:49` uses `py-1.5` + `text-[11px]`, ~28 px tall. WCAG recommends 44 px. Easy fix.

- **Empty state handling is inconsistent.** `TokenBirdsEyeDashboard` has a clean text fallback (lines 63–80); `BirdsEyeDashboard` renders nothing meaningful when there are no pools.
- **Excellent layout intent** with `grid grid-cols-1 lg:grid-cols-[1.4fr_1fr]` at `token-birds-eye-dashboard.tsx:128` — this is the kind of asymmetric grid that makes a dashboard feel designed instead of generated.

RFQ family

Six provider dashboards (`cowswap`, `bebop`, `0x`, `1inch`, `jupz`, plus the depth view) plus a control panel. This is genuinely valuable IP — very few tools surface this. The current organization makes it hard to find unless you know it's there.

- Provider dashboards are sibling routes inside the dashboard view; consider promoting them as a first-class "RFQ" tab in the sidebar with provider switching *inside* the page.
- Provider visual identity (logos, accent color per provider) is missing. Adding a small provider chip in `rfq-header.tsx` would orient the user instantly.

06 Data visualization layer

18 chart components were reviewed. The strongest two or three are *excellent* and could headline the marketing site. The weakest feel hand-rolled and inherit none of the polish from the rest of the design system.

The single biggest finding

> **THREE RENDERING LIBRARIES, THREE CONVENTIONS**
> The chart layer mixes **`lightweight-charts`** (price candlesticks), **Visx** (price spread, LP entry/exit, liquidity history), and **raw SVG + React state** (liquidity distribution, price impact, LP ranges, capital efficiency). Each has its own color conventions, its own tooltip, its own number formatter, and its own opinion about what to do when data is empty. Users feel this as "some charts are nicer than others" without being able to explain why.

Strongest charts

1. **Price Spread Chart** — `components/dashboard/price-spread-chart.tsx`. Semantic colors with intent (red/green only triggered above a 2% deviation threshold), smart hourly/daily axis switching at a 7-day span (line 226), zoom support, crosshair, deviation ranking table, and conditional tooltip styling. This is a reference chart for the rest of the system.
2. **LP Entry/Exit Chart** — `components/dashboard/lp-entry-exit-chart.tsx`. Intentionally avoids red/green (uses teal #5B9A8B for entries, bronze #C49052 for exits) so the chart is colorblind-safe. Dual-scale design separates flow magnitude from active address count.

3. **Lightweight Price Chart** — `components/dashboard/lightweight-price-chart.tsx`. Production-grade candlestick with zoom controls, volume histogram, and a ResizeObserver pattern. Lets down only by hardcoded green/red colors (lines 84, 141–144) instead of theme tokens.

Weakest charts

1. **Liquidity Distribution Chart** (`liquidity-distribution-chart.tsx`) — complex calculation logic mixed directly into rendering, no documented scaling strategy, fragile tooltip positioning, no hover highlight on bars.
2. **Price Impact Chart** (`price-impact-chart.tsx`) — single file is 250+ lines, simulation curve logic not validated, no accessibility labels, no obvious loading state when data is stale.
3. **LP Comparison Table** (`lp-comparison-table.tsx`) — hardcoded green/red on efficiency ratio (lines 34–43), `onClick` without keyboard handler so the table is unusable on keyboard, scrollbar styling not themed.

Cross-chart problems

PROBLEM	WHY IT MATTERS	CITE
Six different number formatters for prices and USD values	Same value shows as `$1.2345` in one chart and `$1.23` in another. Looks careless.	`multiple files`
Hardcoded green/red candle colors	Theme switch can't reach them; colorblind-unsafe.	`lightweight-price-chart.tsx:84,141–144`
No shared Tooltip primitive	Each chart re-implements positioning, styling, escape behavior.	`all charts`
No global time-range selector	Users can't say "show me the last hour everywhere."	`dashboard.tsx`
No CSV / data export	Power users *will* ask for this within their first week.	`all charts`
No annotations / event markers	Can't highlight an incident, an oracle update, an incentive change.	`all charts`
No print / PDF styles	Charts cannot be cleanly exported for reports or screenshots.	`app/globals.css`

07 Loading, empty & error states

A surprising amount of the "feel" of a dashboard product comes from how it behaves when it has nothing to show. Today, Transparent Space is uneven here.

WHAT'S GOOD

- Skeletons exist for both metric cards and chart cards, with proper shimmer/pulse, so initial loads do not jank. `metric-card.tsx:70–85`, `chart-card.tsx:194–212`
- Pool-switch transition uses a branded `SpaceLoader`, which is more memorable than a spinner.
- Some charts (LP entry/exit, liquidity history) handle empty data gracefully. `lp-entry-exit-chart.tsx:117`
- The "space has no pools" case is explicitly designed.

WHAT'S MISSING

- **No global error boundary.** A single chart throwing crashes the whole dashboard tree.
- **Empty states are mostly text.** Most empty states are a centered string. Add an illustration or a single suggested action ("Connect a pool", "Adjust filters").
- **No explicit error states** on data-fetch failure for individual charts — they either show empty or render `null`.
- **Stale data is invisible.** If a chart is showing data from 4 hours ago because the indexer is lagging, the user has no signal. A subtle "last updated 4h ago" chip per card would build trust.
- **Inactive / unauthorized states** for spaces and pools have minimal copy. `app/[spaceSlug]/page.tsx:108–127`

08 Accessibility & touch

The product targets a desk-bound, expert audience, so accessibility is sometimes deprioritized — but the same hygiene that helps a screen reader also helps a power user on a keyboard at 2 am.

- **Focus rings are inconsistent.** The `Button` primitive has good `focus-visible:ring-ring/50` behavior. Most custom interactive elements in the dashboard, sidebar, and tables don't.
- **Missing `aria-current`** on active sidebar items (`sidebar-menu.tsx:37–69`) — a screen reader cannot announce which pool is selected.
- **Token icons have no `alt`** in `pool-header.tsx:158–163`.
- **Touch targets are too small** in birds-eye and token-birds-eye toggles (~28 px tall). WCAG recommends 44 px.
- **Custom buttons via role="button"** in `sidebar-menu.tsx` have keyboard handlers but most other clickable divs across the dashboard do not.
- **Color contrast not validated**. Several charts use `--bp-text-muted` on chart-watermark backgrounds at low alpha — likely below WCAG AA.
- **No skip-to-content** link or landmark `<main>` in the dashboard wrapper.

> **QUICK WIN**
> A single utility class `.bp-focus-ring` applied via the `cn()` helper to every interactive element in the sidebar, tables, and FABs would close 60% of the keyboard accessibility gap in an afternoon.

09 Responsive & cross-device

The app uses Tailwind's standard breakpoints (`sm`, `md`, `lg`) and the sidebar collapses below 768 px. But the experience is still strongly desktop-first.

- **No `max-width` on the dashboard canvas.** On a 32" display, charts stretch to unreadable line lengths.
- **Tablet is a dead zone.** The sidebar collapses, but charts are not laid out for portrait widths — they reflow into a single tall column with no information density advantage.
- **Pool name truncation** on narrow widths is missing.
- **FAB overlays content** on mobile and small windows.
- **Search bar collapses to an indistinguishable icon** on narrow widths.
- **No tested mobile path.** The product is unlikely to ever be primarily mobile, but a usable read-only mobile view (KPI strip + 2–3 most-used charts) would dramatically improve trust during demos.

10 Prioritized recommendations

Effort is rough engineering effort. Impact is opinionated. Items are roughly ordered by *impact ÷ effort* — the first six items are where I would actually start.

#	RECOMMENDATION	IMPACT	EFFORT	PILLAR
1	**Extend Tailwind config with the `--bp-*` tokens.** Expose `bg-bp-card`, `text-bp-accent`, etc., and ban arbitrary `[]` values via lint. Deletes drift at the source.	High	Low	Tokens
2	**Build a single `lib/format-utils.ts`** with `formatPrice`, `formatUsd`, `formatVolume`, `formatPct`. Replace the 6+ ad-hoc formatters across charts.	High	Low	Charts
3	**Centralize chart colors** in `lib/metric-colors.ts` with `CANDLESTICK`, `SPREAD`, `LP_FLOW`, etc. Remove all hex from chart files.	High	Low	Charts
4	**Add a global error boundary** at `DashboardWrapper` and a per-card error fallback. One chart should never take down the dashboard.	High	Low	States
5	**Fix the FAB.** Move it inside the sidebar rail or into `DashboardWrapper` with proper offset. Stop overlaying content.	High	Low	Navigation
6	**Reduce pool-switch loader to ~250 ms** after first load (or skip when cached). Keep the long version only for the first impression.	High	Low	Navigation
7	**Add a global time-range picker** in the dashboard header that fans out to all charts. Single source of truth for "the period I'm looking at."	High	Medium	Charts
8	**Build a shared `<ChartTooltip />` primitive** with positioning, theming, and ARIA. Migrate charts one by one.	Medium	Medium	Charts
9	**Add "last updated" chip** to every chart and metric card, sourced from the data layer. Kills the "is this stale?" trust gap.	High	Medium	States
10	**Truncate long pool names** with tooltip, and constrain dashboard canvas to `max-w-[1680px]`.	Medium	Low	Navigation
11	**Add a breadcrumb component** in `rfq-header.tsx` showing space → pool → provider → view.	Medium	Low	Navigation
12	**Promote RFQ to a first-class section** in the sidebar with provider switching inside the page rather than as sibling routes.	Medium	Medium	Navigation
13	**Standardize empty states** with a small `<EmptyState />` component (icon + headline + optional CTA).	Medium	Low	States

#	RECOMMENDATION	IMPACT	EFFORT	PILLAR
14	**Apply `.bp-focus-ring`** across sidebar, tables, FABs, and tab toggles.	Medium	Low	Accessibility
15	**Bump tab/toggle touch targets** in birds-eye to a minimum of 36–44 px.	Medium	Low	Accessibility
16	**CSV / PNG export** on every chart card via the existing `ChartCardHeader`. Power-user feature, surprisingly cheap if charts share a contract.	Medium	Medium	Charts
17	**Inline help illustrations** in `help-modal.tsx` should move to a per-chart MDX file or be replaced by short looping screen captures.	Low	Medium	Tokens
18	**Pick one theme system**: alias either shadcn tokens or `--bp-*` into the other. Stop maintaining both.	Medium	Medium	Tokens

11 Suggested two-phase roadmap

Phase 1 — Hardening (1–2 sprints)

Pure quality moves with no design risk. Done well, the product visibly "levels up" without anything looking different at first glance.

- Tailwind config extends `--bp-*` tokens and lint forbids arbitrary hex.
- Single `format-utils` + single `metric-colors` module.
- Global error boundary + per-card fallback.
- FAB repositioned, pool name truncation, dashboard `max-width`.
- Pool-switch loader shortened.
- Focus ring + ARIA pass on sidebar / tables / FABs.

Phase 2 — Compounding (2–4 sprints)

Higher-leverage improvements that unlock new product capabilities once the foundation is clean.

- Shared `<ChartTooltip />` primitive and migration of all charts to it.
- Global time-range picker fanning out to all charts via a Zustand slice.
- "Last updated" freshness chip across cards.
- RFQ becomes a first-class section with in-page provider switcher and breadcrumb.
- Standardized `<EmptyState />` + illustrations.
- CSV / PNG export from chart card header.
- Tablet layout pass and a minimal read-only mobile view for demos.

Phase 3 (later) — Differentiators

What turns a hardened tool into a product nobody else has.

- Annotations on time-series charts (oracle updates, incentive changes, incidents).
- Per-user dashboard layouts via `react-grid-layout` (already a dep).
- Saved comparisons across pools / spaces.
- Export to PDF report (this same toolchain), branded.
- A storybook or a public component gallery so design and engineering share a vocabulary.

> **IF YOU ONLY DO ONE THING**
>
> Spend a single sprint on items 1–6 in the priority table. They are all small, they are all independent, and together they remove the three categories of paper cuts a new user notices in their first session: visual drift across components, the FAB overlap, and the "why is this still loading?" feeling. The product will feel *noticeably* more solid without anyone being able to point at exactly what changed — which is the best kind of design work.

Generated from a static audit of the `transparent-space` repository on **2026-04-07**. All citations reference files in the repository at the time of review. This document is intended as an internal working brief; share with care.